

Week5_예습과제

캐글 산탄데르 고객 만족 예측

데이터 세트 : 390개의 피쳐-모두 익명 처리됨, 산탄데르 은행의 고객 만족 여부 예측, 클래스 레이블 명은 TARGET, TARGET=1 : 불만족 고객// TARGET=0 : 만족 고객

모델 성능 평가 : ROC-AUC* $\Rightarrow$ 불만족인 데이터는 일부이기에 정확한 수치보다 더 적합.

*ROC-AUC는 분류 결정 임계값(Threshold)이 변함에 따라 FPR(False Positive Rate)이 변할 때 TPR(True Positive Rate, 재현율)이 어떻게 변하는지 곡선 형태로 나타낸 뒤, 그 아래 면적(AUC)을 구하는 지표, 모델이 양성과 음성을 구분해내는 전반적인 성능을 여러 임계값 기준으로 종합적으로 평가함.

Step1. 데이터 전처리

학습데이터를 DataFrame으로 로딩

- 코드 실행시 발생하는 경고 메시지 화면에 출력되지 않도록
- 지정된 경로에 있는 csv파일을 읽어 판다스의 DataFrame으로 변환한 뒤 cust_df 변수에 저장
 - *경로 앞 r : Raw String 기호, 윈도우 경로 표기법의 역슬래시(\)가 특수문자로 오인되지 않도록 함.
- 불러온 DF의 전체 크기(행 개수, 열개수)를 튜플 형태로 화면에 출력
 - $\Rightarrow$ (76020, 371) $\rightarrow$ 클래스 값 칼럼*을 포함한 피쳐 371개 존재함.
 - *클래스값 칼럼 : 분류 문제에서 모델이 학습하고 예측해야 하는 label을 담고 있는 열(칼럼)
- DF의 맨 앞 3개의 행 데이터를 표 형식으로 출력

전체 데이터에서 만족/불만족 비율(레이블 비율) 파악 $\Rightarrow$ 4%

DF의 describe() 메서드로 각 피처의 값 분포 확인(출력), 출력에 따라 별도의 DF로 수정해 저장.

- var3 칼럼의 경우 min 값이 -999999 → NaN이나 특정 예외 값을 -999999로 변환한 것. -999999는 다른 값에 비해 편차가 너무 심하므로 -999999를 var3에서 가장 값이 많은 2로 변환
- ID 피처는 단순 식별자에 불과하므로 피처를 드롭함
- 클래스(=레이블) 데이터 세트와 피처 데이터 세트를 분리해 별도의 데이터 세트로 별도로 저장

⇒ 새로운 DF의 shape 출력 결과, 칼럼이 기존의 371에서 369개 됨.

학습과 성능 평가를 위해 원본 데이터 세트에서 학습 데이터, 테스트 데이터 세트를 각각 분리함. 이때 Target 값 분포가 학습 데이터와 테스트 데이터 세트에 모두 비슷하게 추출됐는지 확인

- train_test_split()함수로 학습용 테스트용으로 데이터 세트 분리, 이때 전체 피처 데이터인 X_features, 정답 레이블인 y_labels를 학습용, 테스트용으로 분할, test_size=0.2로 테스트 데이터 비율을 전체의 20%로, random_state=0로 난수 발생 시드를 고정해 코드를 다시 실행해도 똑같은 방식으로 데이터가 섞이고 나뉘지도록

⇒ 학습, 테스트 데이터 세트 모두 레이블 값의 분포가 원본 데이터와 유사하게 전체 데이터의 4% 정도의 불만족(1) 값으로 만들어짐을 알 수 있음.

XGBoost의 조기 중단*의 검증 데이터 세트로 사용하기 위해 X_train, y_train을 다시 쪼개서 학습/검증 데이터 세트를 만듦

*조기중단을 사용해 과적합 예방, 검증 데이터는 모델이 새로운 데이터에도 잘 통하는 진짜 실력을 가지고 있는지 객관적으로 평가하여, 가장 최적의 학습 중단 타이밍을 잡아주는 심판 역할의 데이터

⇒데이터 세트 정리:

X_train (학습용 피처 / 특징)

- 역할: 모델이 패턴을 공부하기 위해 사용하는 특징들

y_train (학습용 레이블 / 정답)

- 역할: 모델이 문제를 풀고 맞았는지 틀렸는지 확인할 수 있도록 제공되는 정답지

- `x_tr`, `y_tr` (학습 데이터) : 모델이 패턴을 '공부'하는 용도
- `x_val`, `y_val` (검증 데이터) : 모델이 공부를 잘하고 있는지 중간 성적을 채점하는 용도

`x_test` (테스트용 피쳐 / 특징)

- 역할: 모델의 진짜 실력을 평가하기 위해 남겨둔 한 번도 보지 못한 새로운 특징

`y_test` (테스트용 레이블 / 정답)

- 역할: 테스트 문제의 진짜 정답지. 모델이 `x_test` 를 보고 예측한 결과와 이 `y_test` 를 비교하여 모델의 성능(정확도 등)을 최종 평가하게 됨.

Step2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

XGBoost의 학습 모델 생성, 예측 결과를 ROC AUC로 평가

- 조기 중단(`early_stopping_rounds`)는 100으로 설정
- 교재와 달리 최근 버전의 XGBoost API로 발생하는 호환성에 따라 `early_stopping_rounds`, `eval_metric`을 모델 생성하는 객체 선언 시에 넣기

```
XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156, early_stopping_rounds=100, eval_metric="auc")
```

HyperOpt를 이용해 베이지안 최적화 기반으로 XGBoost의 하이퍼 파라미터 튜닝*을 수행

- `max_depth` : 5~15까지, 1 간격으로
- `min_child_weight` : 1~6까지, 1 간격으로
- `colsample_bytree` : 0.5~0.95
- `learning_rate` : 0.01~0.2 사이 정규 분포된 값으로 하이퍼 파라미터 검색 공간 설정

목적 함수 생성 → 3 Fold 교차 검증을 이용해 평균 ROC-AUC 값을 반환하되 -1을 곱해주어 최대 ROC-AUC 값이 최소 반환값이 되도록

- 교차 검증 시 KFold 클래스를 이용해 직접 학습, 검증 데이터를 추출하고 이를 교차 검증 횟수만큼 학습과 성능 평가를 수행* → XGBoost의 조기 중단과 검증 데이터 성능 평가

- 수행 시간을 줄이기 위해 estimators는 100으로 줄이고, early_stopping_rounds도 30으로 줄여 테스트한 뒤 나중에 하이퍼 파라미터 튜닝이 완료되면 다시 증가시키기

fmin()함수를 호출해 max_eval=50회만큼 반복하면서 최적의 하이퍼 파라미터 도출

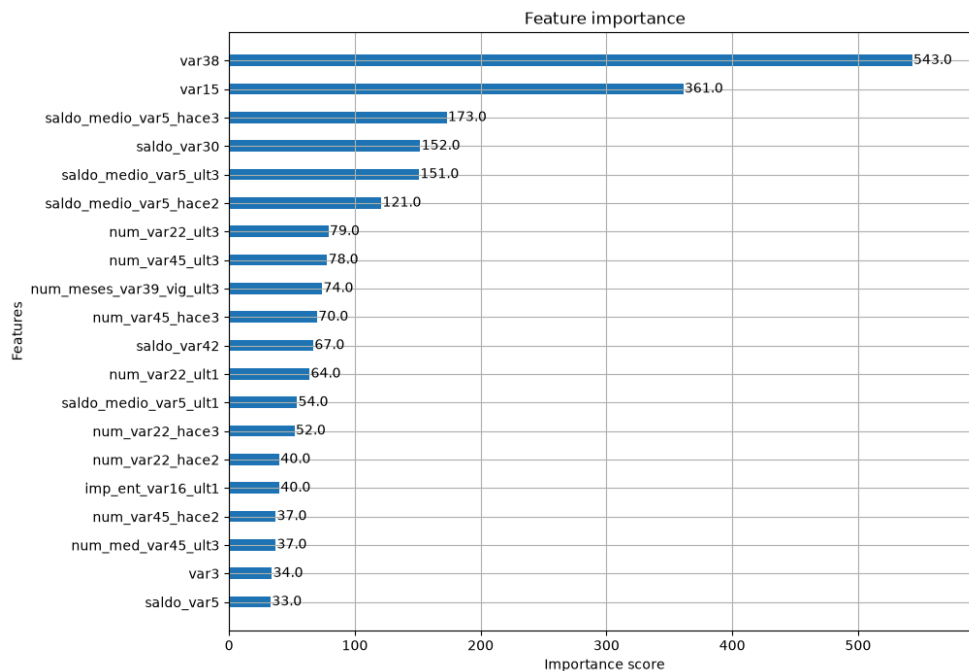
⇒best: {'colsample_bytree': np.float64(0.616616729092767), 'learning_rate': np.float64(0.04328838086931633), 'max_depth': np.float64(6.0), 'min_child_weight': np.float64(6.0)}

이렇게 도출된 최적 하이퍼 파라미터를 기반으로 XGBClassifier 재학습 시키고 테스트 데이터 세트에서 ROC AUC를 측정

- 위에서 줄였던 n_estimators는 500으로 다시 증가

⇒ ROC AUC가 이전 예제에 비해 하이퍼 파라미터 튜닝 이후 개선됨.

튜닝된 모델에서 각 피처의 중요도를 피처 중요도 그래프로 나타내기 → xgboost 모듈의 plot_importance()메소드 이용



⇒ XGBoost의 예측 성능을 좌우하는 가장 중요한 피처들은 var38, var15

Step3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

앞서 만든 데이터 세트를 기반으로 LightGBM으로 학습 수행, ROC-AUC를 측정하기

- xgboost보다 학습 시간 단축

HyperOpt를 이용해 다양한 하이퍼 파라미터에 대한 튜닝 수행

- 튜닝 대상 : num_leaves, max_depth, min_child_samples, subsample, learning_rate / 이를 이한 하이퍼 파라미터 검색 공간을 설정함.

목적 함수 생성 ~ 앞 XGBoost와 크게 다르지 않음

fmin()을 호출해 최적 하이퍼 파라미터 도출

⇒

```
best: {'learning_rate': np.float64(0.02347777899863792), 'max_depth':  
np.float64(134.0), 'min_child_weight': np.float64(75.0), 'num_leaves':  
np.float64(34.0), 'subsample': np.float64(0.7754775757159653)}
```

최적 하이퍼 파라미터를 이용해 LightGBM 학습 후에 테스트 데이터 세트에서 ROC-AUC를 평가